C PROGRAMMING – COMPLETE ASSIGNMENT ANSWERS

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

Tasks covered: Task 1 – Task 4

Student Name: _____ **Register No:** _____

Compiler used: GCC (gcc) **Language:** C

Scenario summary

Grama Jyothi Rural Electricity Co-operative bills about 4,000 households every month. The billing program reads previous and current meter readings, calculates consumption, applies slab-based energy charges, and must correctly handle a 16-bit meter rollover from 65535 to 0. The design should also be suitable for growth from a village to a district-scale system.

TASK 1 – Organise the Program with Header Files and Conditional Compilation (3 Marks)

Objective: Organise constants and function prototypes in a header file and provide a compile-time DEBUG mode.

1. Header file: billing.h

```
#ifndef BILLING_H
#define BILLING_H

#include <stdio.h>
#include <stdint.h>

#define METER_MAX 65535U
#define FREE_UNITS 100U
#define SECOND_SLAB_UNITS 100U

#define RATE_FIRST 0.0
#define RATE_SECOND 2.25
#define RATE_THIRD 4.50

#define DEBUG_PRINT(...) \
    do { \
        if (0) { printf(__VA_ARGS__); } \
    } while (0)

uint32_t calculate_units(uint16_t previous, uint16_t current);
double calculate_energy_charge(uint32_t units);
void print_bill(uint16_t previous, uint16_t current);

#endif
```

2. Main program: billing.c

```
#include "billing.h"

#ifdef DEBUG
#undef DEBUG_PRINT
#define DEBUG_PRINT(...) \
    do { printf("[DEBUG] "); printf(__VA_ARGS__); printf("\n"); } while (0)
#endif

uint32_t calculate_units(uint16_t previous, uint16_t current)
{
    if (current >= previous)
        return (uint32_t)current - previous;
```

```
    /* Meter rolled over from 65535 to 0. */
    return (uint32_t)(METER_MAX - previous + 1U) + current;
}

double calculate_energy_charge(uint32_t units)
{
    if (units <= FREE_UNITS)
        return 0.0;

    if (units <= FREE_UNITS + SECOND_SLAB_UNITS)
        return (units - FREE_UNITS) * RATE_SECOND;

    return SECOND_SLAB_UNITS * RATE_SECOND
           + (units - FREE_UNITS - SECOND_SLAB_UNITS) * RATE_THIRD;
}

void print_bill(uint16_t previous, uint16_t current)
{
    uint32_t units = calculate_units(previous, current);
    double charge = calculate_energy_charge(units);

    DEBUG_PRINT("Previous=%u Current=%u Units=%u",
                previous, current, units);

    printf("Previous reading : %u\n", previous);
    printf("Current reading  : %u\n", current);
    printf("Units consumed   : %u\n", units);
    printf("Energy charge    : Rs %.2f\n", charge);
}

int main(void)
{
    uint16_t previous, current;

    printf("Enter previous meter reading: ");
    if (scanf("%hu", &previous) != 1 || previous > METER_MAX) {
        printf("Invalid previous reading.\n");
        return 1;
    }

    printf("Enter current meter reading: ");
    if (scanf("%hu", &current) != 1 || current > METER_MAX) {
        printf("Invalid current reading.\n");
        return 1;
    }

    print_bill(previous, current);
    return 0;
}
```

Why include guards? The #ifndef/#define/#endif guard prevents the same header from being included more than once in a compilation unit.

Conditional compilation: With normal compilation, DEBUG code is excluded. With **-DDEBUG**, debugging messages are enabled at compile time.

Compile: `gcc billing.c -o billing`

Debug compile: `gcc -DDEBUG billing.c -o billing_debug`

Alternative concept: If debugging were controlled by a run-time variable, every DEBUG_PRINT call would require a condition check during execution. Compile-time selection can remove the debugging code path from the normal build, while a run-time flag is more flexible but may add repeated branch checks.

Evidence expected: Header file, working normal build, working DEBUG build, and explanation of the program structure.

TASK 2 – Read and Print Data Correctly (4 Marks)

Objective: Use the correct scanf/printf format specifiers, validate scanf return values, and print the bill accurately.

Correct format specifiers

C type	scanf	printf
int	%d	%d
unsigned int	%u	%u
long	%ld	%ld
unsigned long	%lu	%lu
float	%f	%f / %.2f
double	%lf	%f / %.2f
char	%c	%c
string	%s	%s

Example input validation

```
uint16_t previous, current;

if (scanf("%hu", &previous) != 1) {
    printf("Invalid input.\n");
    return 1;
}

if (scanf("%hu", &current) != 1) {
    printf("Invalid input.\n");
    return 1;
}
```

What happens with a wrong format specifier? Using a format specifier that does not match the object being read can cause undefined behaviour. For example, a long integer should not be read using a format intended for a different type. The correct specifier should match the variable type.

Detecting invalid input: scanf returns the number of successfully assigned input items. Therefore, checking `scanf(...) != expected_count` detects letters, missing values, or other invalid input.

Printing rupees and paise: Store money internally as an integer number of paise where practical. For example, Rs 162.50 equals 16250 paise. When displaying it, divide by 100 for rupees and use the remainder for paise.

```
long long paise = 16250;
printf("Rs %lld.%02lld\n", paise / 100, paise % 100);
```

Alternative concept: If an entire line were read with fgets() and then converted using strtol()/strtoul(), the program could inspect the whole line and identify trailing or unexpected characters more precisely. Direct scanf is shorter for controlled input, while fgets + conversion generally provides stronger input validation.

Evidence expected: Correct format specifiers, input validation using scanf's return value, and a correctly formatted bill.

TASK 3 – Control Type Conversion and Precision (4 Marks)

Objective: Identify implicit/explicit conversions, integer division problems, and rounding issues in billing calculations.

Important conversion points

• **Integer promotion:** Smaller integer types such as char and short are commonly promoted to int in arithmetic expressions.

• **Usual arithmetic conversions:** When operands have different types, C converts them according to its conversion rules before performing the operation.

• **Integer division:** 5 / 2 gives 2 when both operands are integers. It does not give 2.5.

• **Explicit casting:** (double)units / 100.0 forces floating-point arithmetic.

• **Money precision:** Representing money as integer paise avoids many binary floating-point rounding problems.

Three unexpected expressions

```
/* Surprise 1: integer division */
printf("%.2f\n", 5 / 2);          /* prints 2.00 */

/* Correct */
printf("%.2f\n", 5.0 / 2);        /* prints 2.50 */

/* Surprise 2: integer truncation */
int a = 7, b = 3;
printf("%d\n", a / b);             /* 2 */

/* Correct if fractional result is required */
printf("%.2f\n", (double)a / b);   /* 2.33 */

/* Surprise 3: integer to floating conversion happens too late */
int units = 125;
double charge = (units / 100) * 2.25;  /* units/100 = 1 */
printf("%.2f\n", charge);              /* 2.25 */

/* Better */
double charge2 = (units / 100.0) * 2.25; /* 2.8125 */
printf("%.2f\n", charge2);
```

Consistent rounding rule: Keep the monetary value in paise as an integer whenever possible. If a fractional charge must be converted to paise, apply one clearly documented rounding rule, such as rounding to the nearest paise, rather than repeatedly rounding intermediate values.

Alternative concept and complexity: If all calculations were performed in floating point and rounded only once at the end, intermediate rounding error would be reduced. However, floating-point operations can introduce representation error. Integer-paise arithmetic is deterministic for currency. Both approaches remain O(1) per bill because each bill uses a fixed number of arithmetic operations; the main difference is precision and constant-factor cost.

Evidence expected: Identified conversion points, three demonstrated surprises with explanations, and a consistent rounding rule.

TASK 4 – Trace the Compilation Process (4 Marks)

Objective: Follow a C program through preprocessing, compilation, assembly, object generation, and linking.

1. Preprocessing

Command: `gcc -E billing.c -o billing.i`

The preprocessor expands #include files, processes #define macros, and evaluates conditional compilation directives such as #ifdef and #if. The result is preprocessed C source code.

2. Compilation to assembly

Command: `gcc -S billing.c -o billing.s`

The compiler translates the C program into assembly language for the target architecture. This stage also performs language and semantic checks and optimization when enabled.

3. Object file

Command: `gcc -c billing.c -o billing.o`

The assembler creates an object file containing machine code and relocation/symbol information. External references may still need to be resolved by the linker.

4. Linked executable

Command: `gcc billing.o -o billing`

The linker combines object code and required libraries, resolves symbols, performs relocation, and creates the final executable.

Types of errors

Error type	Example	When it occurs
Compile-time error	Missing semicolon or invalid C syntax	During compilation
Link-time error	Calling a function whose implementation is not linked	During linking
Run-time error	Invalid operation or unexpected input causing failure	While executing

Useful warnings and fixes

Compile with warnings enabled: `gcc -Wall -Wextra -std=c11 billing.c -o billing`. Warnings such as unused variables, missing declarations, or suspicious conversions should be investigated rather than ignored. Typical fixes include using the correct format specifier, adding the required header, initializing variables, and checking function return values.

Alternative concept and complexity analysis: If the program were split into input.c, tariff.c, and report.c with a shared header, only the changed source file generally needs recompilation before relinking. In a single-file build, a change to any part causes the whole source file to be recompiled. For a small program the difference is negligible, but for a large project separate compilation can substantially reduce rebuild time. Object files also consume additional disk space, but they enable modular builds and clearer project organisation.

Complete test cases

Previous	Current	Units	Charge
1000	1050	50	Rs 0.00
1000	1150	150	Rs 112.50
1000	1300	300	Rs 675.00
65530	5	11	Rs 0.00

Note: The sample slab calculation follows the rates shown in the assignment screenshot: first 100 units free, next 100 units at Rs 2.25/unit, and units above 200 at Rs 4.50/unit.

Submission checklist

■ Create the GitHub repository using the required naming format.

■ Keep each task in its own folder (Task01, Task02, Task03, Task04, etc.).

■ Include source code and screenshots/evidence for each task.

■ Add README.md containing your name, register number, compiler version, compile commands, run steps, and brief results.

■ Commit work progressively, ideally at least one commit per task.

■ Submit the working GitHub repository link in the lab portal and on the report cover page.